

# NLP Patient Description to Acuity

Deep Learning and Mathematical Triage

Quaigraine Samuel    Twum Samuel

Department of Mathematics  
Kwame Nkrumah University of Science and Technology

June 27, 2026

# Outline

- 1 Clinical Context
- 2 NLP Pipeline
- 3 BERT Pre-Training
- 4 Learning Math
- 5 Summary

# The Clinical Scenario



Immediate / special

Red

Blue

Orange

Yellow

Green

Blue = deceased on arrival  
(dignity protocol, not low urgency)

Decreasing urgency (living patients) →

**Setting:** Komfo Anokye Teaching Hospital (KATH), Kumasi — **SATS** triage.

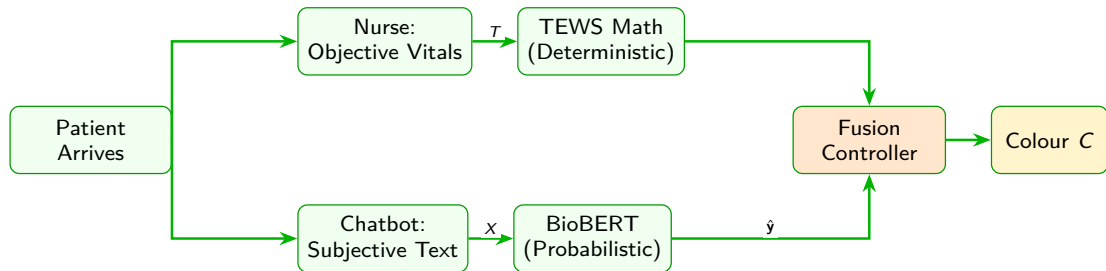
**Chief complaint (running example):**

*"I have a headache and feel feverish. My body aches and I feel weak."*

How does a computer mathematically interpret this simple, unstructured complaint and convert it into a triage colour?

*The system supports triage staff; it does not diagnose disease or replace clinical judgment.*

# The Dual Pathway: TEWS vs. NLP Chatbot

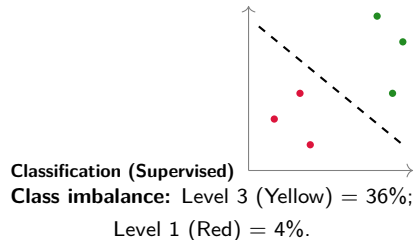


- **TEWS:** Deterministic sum of vital-sign points from SATS tables:  $T = \sum_k w_k f_k(v_k)$ .
- **NLP:** Probabilistic classifier on chief-complaint text — catches symptoms before vitals are measured.
- **Safety rule:** Fusion never assigns a less urgent colour than a strong language signal requires.

# NLP Analysis: Supervised Deep Learning



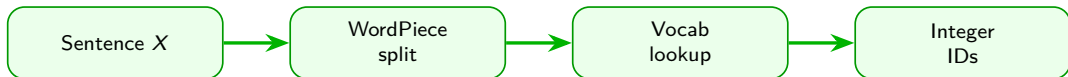
- **Supervised learning** on  $\sim 80,000$  nurse-labeled chief complaints.
- **Output:** Probabilities over 5 acuity levels  $\rightarrow$  SATS colours (L1–2 Red/Orange; L3 Yellow; L4–5 Green).
- Bag-of-words fails: “I do *not* have a fever” vs. “I *do* have a fever” look identical to a word counter.



# Tokenization: Raw Text to Integer IDs

Computers cannot read English — they only understand numbers.  $X \xrightarrow{\tau} (t_1, \dots, t_M)$ ,  $M \leq 128$ .

- **WordPiece**: common words whole; rare words split (“headache”  $\rightarrow$  head+##ache)
- Vocabulary  $\mathcal{V}$ :  $\sim 30,000$  entries; each ID = row index in embedding table



# Worked Example: Full ID Assignment

**Input:** “I have a headache and feel feverish.” Each ID  $\rightarrow$  row in  $E_{\text{word}}$ . Real tokens:  $M \approx 12$ ; batches pad to length 128 with filler tokens.

| $i$ | Raw      | Token        | ID         |
|-----|----------|--------------|------------|
| 0   | —        | [CLS]        | 101        |
| 1   | I        | i            | 1045       |
| 2   | have     | have         | 2031       |
| 3   | a        | a            | 1037       |
| 4   | headache | head+###ache | 7994, 8772 |
| 5   | and      | and          | 1998       |

| $i$      | Raw      | Token        | ID         |
|----------|----------|--------------|------------|
| 6        | feel     | feel         | 2514       |
| 7        | feverish | fever+###ish | 9643, 6804 |
| 8        | .        | .            | 1012       |
| 9        | —        | [SEP]        | 102        |
| 10       | —        | [PAD]        | 0          |
| 11       | —        | [PAD]        | 0          |
| $\vdots$ | $\vdots$ | $\vdots$     | $\vdots$   |
| 127      | —        | [PAD]        | 0          |

**Vocabulary range:** token IDs  $0 \dots 28,995$  ( $|\mathcal{V}| = 28,996$ ); sequence positions  $i = 0 \dots 127$  ( $M \leq 128$ ).

| Special token | ID  | Role                                                     |
|---------------|-----|----------------------------------------------------------|
| [PAD]         | 0   | Padding filler after [SEP]; attention mask = 0 (ignored) |
| [UNK]         | 100 | Unknown WordPiece piece                                  |
| [CLS]         | 101 | Classification summary (always $i = 0$ )                 |
| [SEP]         | 102 | Sentence boundary                                        |
| [MASK]        | 103 | MLM pre-training only — not used at triage inference     |

# What is [CLS] ?

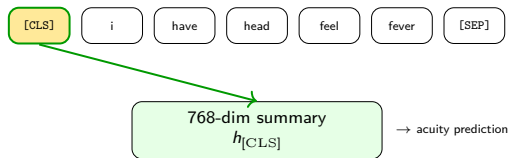
## [CLS] = Classification token

- Artificial token — *not* from the patient's words
- Always inserted at position  $i = 0$  before any real text
- **Purpose:** After 12 transformer layers, its hidden state  $h_{[\text{CLS}]}$  becomes a 768-dimensional **summary** of the entire complaint
- This summary vector is multiplied by classification weights to predict acuity

[SEP] (ID 102) marks the end of the sentence — a boundary marker.

### In plain English

Think of [CLS] as an empty notebook at the start. As the model reads every word, it fills that notebook with a compressed summary used for the final triage decision.





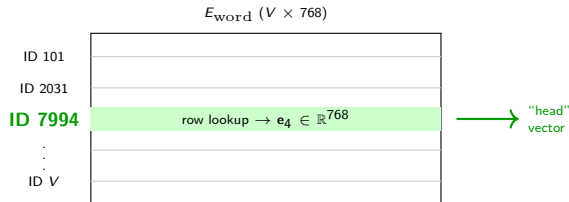
# Embedding Lookup: ID to 768-D Vector

A token ID alone carries no meaning. The **embedding layer** converts each ID to a dense vector:

$$\mathbf{e}_i = E_{\text{word}}[\text{id}(t_i)] \in \mathbb{R}^{768}$$

## Variables:

- $V \approx 30,000$  — vocabulary size (rows)
- $D = 768$  — embedding dimension (columns)
- $E_{\text{word}} \in \mathbb{R}^{V \times 768}$  — learned weight matrix
- ID 2031  $\rightarrow$  row 2031  $\rightarrow$  vector  $[0.04, -0.21, \dots, 0.17]$



## In plain English

The embedding table is like a dictionary where each word ID points to a list of 768 numbers that encode its meaning.

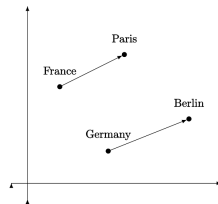
# What the Vector Represents

Each of the 768 dimensions is a **learned feature** — not hand-designed.

- Similar words have similar vectors (close in space)
- “headache”, “feverish”, “aches” cluster near “pain”, “fever” in BioBERT’s PubMed-trained space
- **Static** (Word2Vec): one vector per word forever
- **Contextual** (BERT): same word, different vector depending on sentence — next slide

Example vector for “headache”:

$\mathbf{e} = [0.12, -0.45, 0.88, \dots, -0.01]$  (768 numbers)

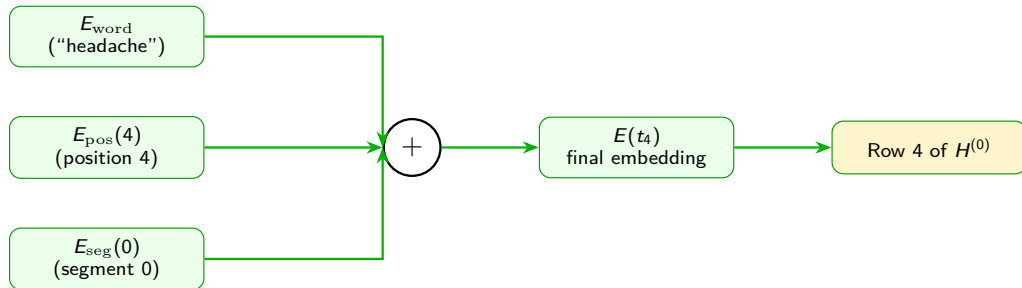


Word vectors in semantic space.<sup>a</sup>

---

<sup>a</sup>Word embedding illustration, Fschwarzentruber, Wikimedia Commons.

# Contextual Mapping: Three Embeddings Summed



Three separate learned vectors are **added element-wise** for each token position  $i$  to produce the input embedding  $E(t_i)$ .

# Contextual Mapping: Formula Breakdown

$$E(t_i) = E_{\text{word}}(t_i) + E_{\text{pos}}(i) + E_{\text{seg}}(t_i)$$

| Term                | Meaning                              | Example                        |
|---------------------|--------------------------------------|--------------------------------|
| $E_{\text{word}}$   | Token identity — what word is this?  | “headache” vs “feverish”       |
| $E_{\text{pos}}(i)$ | Position in sentence — order matters | “head ache” $\neq$ “ache head” |
| $E_{\text{seg}}$    | Which sentence (A or B)              | 0 for our single complaint     |

**Bidirectional reading:** BERT sees all tokens left *and* right — “feel feverish” informs “headache” in both directions.

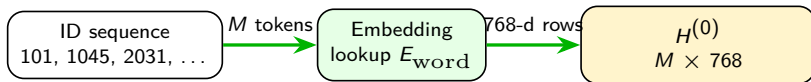
*The word “feel” at position 6 gets a different  $E_{\text{pos}}$  than at position 12 in a longer sentence.*

# The Input Matrix $H^{(0)}$

After embedding every token, the full sentence becomes a matrix:

$$H^{(0)} \in \mathbb{R}^{M \times 768}$$

Each **row** = one token's 768-dimensional embedding. Each **column** = one feature dimension across all tokens.



Row 0 = [CLS] highlighted → becomes triage summary

The matrix shape stays  $M \times 768$  through all 12 layers — only the *values* inside change.

# Matrix Walkthrough with Numbers

**Example submatrix** (first 4 of 768 dimensions shown):

| $i$      | Token    | $D_1$    | $D_2$    | $D_3$    | $D_4$    | $\dots$  |
|----------|----------|----------|----------|----------|----------|----------|
| 0        | [CLS]    | 0.02     | 0.11     | -0.05    | 0.33     | $\dots$  |
| 1        | i        | -0.08    | 0.04     | 0.19     | 0.01     | $\dots$  |
| 2        | have     | 0.15     | -0.22    | 0.07     | 0.44     | $\dots$  |
| 4        | head     | 0.31     | -0.18    | 0.52     | 0.09     | $\dots$  |
| 7        | fever    | 0.44     | 0.27     | -0.11    | 0.61     | $\dots$  |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |

After  $L = 12$  transformer layers:  $h_{[\text{CLS}]} = H_{0,:}^{(12)}$  — the first row of the final layer output, a single 768-dim vector summarising the complaint.

# Diagram: 12 Stacked Encoder Layers

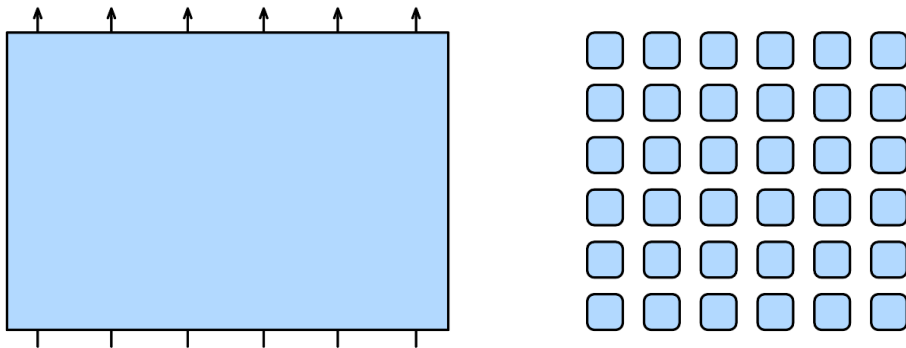


Diagram: Zhang et al., d2l-ai, CC BY-SA 4.0. Each block = one encoder layer (self-attention + feed-forward).

# Inside One Encoder Layer: Formulas

Each layer  $\ell = 1, \dots, 12$  performs two sub-steps with **residual connections**:

$$Z^{(\ell)} = \text{LayerNorm}(H^{(\ell-1)} + \text{MultiHeadAttention}(H^{(\ell-1)}))$$

$$H^{(\ell)} = \text{LayerNorm}(Z^{(\ell)} + \text{FFN}(Z^{(\ell)}))$$

| Component          | What it does                                                               |
|--------------------|----------------------------------------------------------------------------|
| MultiHeadAttention | 12 parallel self-attention heads; each learns different word relationships |
| FFN                | Two linear layers: $\text{FFN}(x) = W_2 \text{ReLU}(W_1 x + b_1) + b_2$    |
| LayerNorm          | Normalises activations — stabilises training across 12 layers              |
| Residual (+)       | Adds input back to output — prevents information loss in deep stacks       |



# Stacking 12 Layers: From Words to Clinical Understanding

$$H^{(0)} \xrightarrow{\text{Layer 1}} H^{(1)} \xrightarrow{\text{Layer 2}} \dots \xrightarrow{\text{Layer 12}} H^{(12)} = H^{(L)}$$

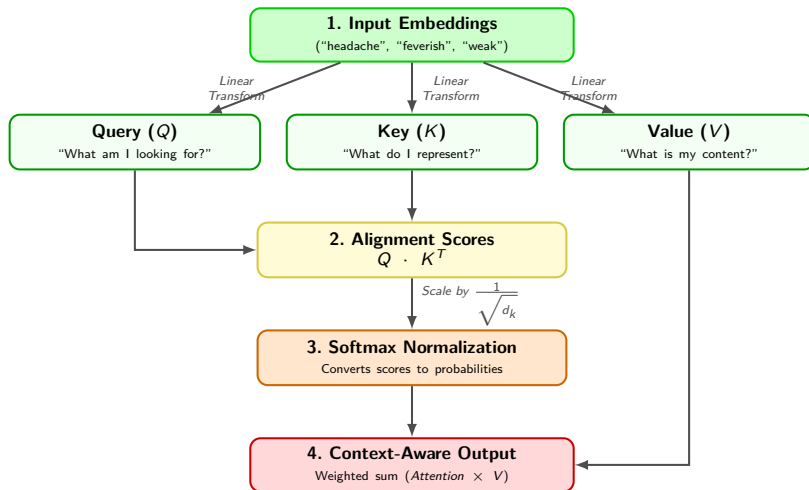
- **Shallow layers** (1–4): recognise individual words and simple phrases
- **Middle layers** (5–8): combine symptoms — “headache” + “feverish” + “weak”
- **Deep layers** (9–12): abstract clinical urgency patterns
- **Parallel processing**: all  $M$  tokens processed simultaneously (not left-to-right)

**Output:**  $h_{[\text{CLS}]} = H_{0,:}^{(L)} \in \mathbb{R}^{768} \rightarrow$  fed to the classification head.

## In plain English

Stacking layers is like reading a complaint once for words, once for phrases, and once for overall urgency.

# Self-Attention Mechanism



# Self-Attention Equation: Variable by Variable

From Vaswani et al. (2017):

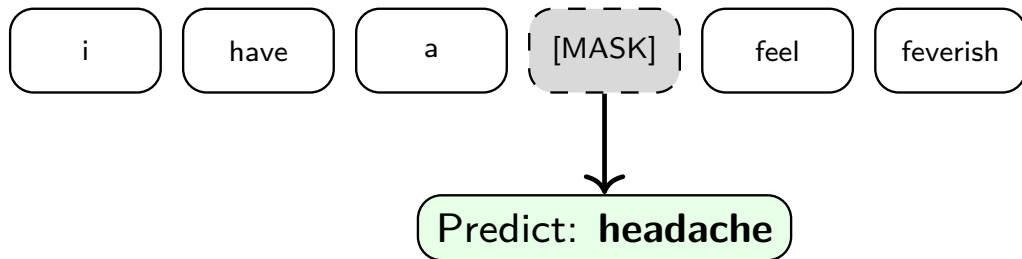
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

| Symbol       | Definition                      | Role in triage                                    |
|--------------|---------------------------------|---------------------------------------------------|
| $Q$          | Query matrix = $HW_Q$           | "What is this token looking for?"                 |
| $K$          | Key matrix = $HW_K$             | "What does each token offer?"                     |
| $V$          | Value matrix = $HW_V$           | Semantic content passed forward                   |
| $d_k$        | Key dimension (= 64 per head)   | Scaling factor                                    |
| $QK^{\top}$  | Pairwise relevance scores       | "headache" $\leftrightarrow$ "feverish" co-attend |
| $\sqrt{d_k}$ | Prevents dot products exploding | Training stability                                |

**Worked weights** for query "headache": feverish 0.41, headache 0.32, weak 0.18, filler words  $\approx$  0.02.

# BERT Pre-Training: Masked Language Modeling

Before triage fine-tuning, BioBERT learns medical English from PubMed via **Masked Language Modeling (MLM)**:



- 15% of tokens randomly masked; model predicts them from **bidirectional** context
- Teaches medical vocabulary ("feverish", "headache", "tachycardia") before any triage labels
- BioBERT = BERT pre-trained on PubMed + PMC biomedical abstracts

# MLM Loss Formula

## Masked Language Modeling Loss

$$\mathcal{L}_{\text{LM}} = - \sum_{i \in \mathcal{M}} \log P(t_i \mid H^{(L)}; \theta)$$

| Variable                  | Meaning                                                   |
|---------------------------|-----------------------------------------------------------|
| $\mathcal{M}$             | Set of masked token positions in the sentence             |
| $t_i$                     | The true (correct) word at masked position $i$            |
| $H^{(L)}$                 | Final hidden states from all 12 layers — the context      |
| $\theta$                  | All model parameters (weights + biases)                   |
| $P(t_i \mid \cdot)$       | Softmax probability the model assigns to the correct word |
| $\mathcal{L}_{\text{LM}}$ | Total penalty — lower = better word predictions           |

# Independent vs. Dependent Variables in MLM

- **Independent variable** ( $H^{(L)}$ ): The surrounding, unmasked context the model is allowed to read bidirectionally
- **Dependent variable** ( $\mathcal{L}_{LM}$ ): The loss penalty — its size depends entirely on  $P(t_i)$  the model assigned to the correct hidden word

## Numerical example:

- Correct word = “headache”; model assigns  $P = 0.92 \Rightarrow \text{contribution} = -\log(0.92) \approx 0.08$  (small penalty)
- Model assigns  $P = 0.01$  to correct word  $\Rightarrow -\log(0.01) \approx 4.6$  (large penalty)

## In plain English

After MLM pre-training on PubMed, BioBERT is fine-tuned on 80,000 triage-labeled chief complaints using cross-entropy loss — covered next.

# Attention Softmax: The Probability Filter

To assign mathematical weight to each word, raw scores are normalised:

$$\alpha_j = \frac{\exp(e_j)}{\sum_{k=1}^m \exp(e_k)}$$

| Variable           | Meaning                                                  |
|--------------------|----------------------------------------------------------|
| $e_j$              | Raw attention score for token $j$ (before normalisation) |
| $\exp(e_j)$        | Forces positive values; amplifies large scores           |
| $\sum_k \exp(e_k)$ | Normalising constant — ensures all weights sum to 1      |
| $\alpha_j$         | Final attention weight for token $j$ (a probability)     |
| $m$                | Sequence length (number of tokens)                       |

**Effect:** “I”, “a”, “and”  $\rightarrow \alpha \approx 0.01$ ; “headache”, “feverish”  $\rightarrow \alpha \approx 0.35+$ .

# Classification Softmax: Acuity Probabilities

The summary vector is mapped to triage class probabilities:

$$\hat{\mathbf{y}} = \text{softmax}(W h_{[\text{CLS}]} + \mathbf{b}), \quad \hat{y}_c = P(\text{acuity} = c \mid X)$$

| Variable                          | Meaning                                             |
|-----------------------------------|-----------------------------------------------------|
| $h_{[\text{CLS}]}$                | 768-dim summary vector from final layer             |
| $W \in \mathbb{R}^{5 \times 768}$ | Learned classification weight matrix                |
| $\mathbf{b} \in \mathbb{R}^5$     | Per-class bias terms                                |
| $\hat{y}_c$                       | Probability of acuity level $c \in \{1, \dots, 5\}$ |
| $\sum_c \hat{y}_c = 1$            | Valid probability distribution over 5 classes       |

## Worked example — our running complaint

*"I have a headache and feel feverish. My body aches and I feel weak."*

Level 3 (Moderate): **72%**  $\Rightarrow$  maps to **Yellow**



# Forward Pass and Cross-Entropy Loss

## 1. Forward Pass

$y = Wx + b$  where  $x = h_{[\text{CLS}]}$ ,  $y$  = predicted logits,  $W$  and  $b$  are learned.

## 2. Cross-Entropy Loss (The Penalty)

$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$  where  $C = 5$  acuity classes.

| Symbol        | Meaning                                                    |
|---------------|------------------------------------------------------------|
| $x$           | Input vector ( $h_{[\text{CLS}]}$ — the complaint summary) |
| $W, b$        | Weights and bias the model must learn                      |
| $y$           | True one-hot nurse label (e.g. Level 3 = Yellow)           |
| $\hat{y}$     | Model's predicted probability distribution                 |
| $\mathcal{L}$ | Loss — 0 if perfect; large if wrong with high confidence   |

Example: true label = Yellow, model predicts Green at 90%  $\Rightarrow \mathcal{L} = -\log(0.05) \approx 3.0$ .

# Backpropagation: The Chain Rule

## 3. Backpropagation (The Investigation)

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial W}$$

- Uses the **chain rule** of calculus to travel backward through all 12 layers + classification head
- $z = Wh + b$  — pre-softmax logits (raw scores before probability conversion)
- Each partial derivative pinpoints which specific weight caused the bad guess
- Gradients flow from the loss at the top back to every embedding and attention weight

### In plain English

If the model confidently assigns 90% to Green when the nurse labeled Yellow, backprop finds exactly which weights in layers 8–12 over-weighted non-urgent patterns.

# Gradient Descent and Fine-Tuning

## 4. Optimization (The Correction)

$$W_{\text{new}} = W_{\text{old}} - \alpha \frac{\partial \mathcal{L}}{\partial W}$$

| Variable                            | Meaning                                                            |
|-------------------------------------|--------------------------------------------------------------------|
| $\alpha$                            | Learning rate (step size; e.g. $2 \times 10^{-5}$ )                |
| $\partial \mathcal{L} / \partial W$ | Gradient — direction of steepest loss <i>increase</i>              |
| Minus sign (—)                      | Move <i>opposite</i> to gradient — go downhill on the loss surface |

**Fine-tuning loss** on triage data:

$$\mathcal{L}_{\text{triage}} = - \sum_{i=1}^N \log P(y_i | X_i; \theta), \quad N \approx 80,000$$

Best eval\_loss = 0.001812 after 3 epochs (13,500 steps) on nurse-labeled chief complaints.

# End-to-End: Complaint to Triage Colour

**Putting it all together** for our running example:

- 1 Patient says: *"I have a headache and feel feverish. My body aches and I feel weak."*
- 2 Tokenize → embed → 12 transformer layers
- 3 Self-attention weights "headache", "feverish", "weak" heavily
- 4  $h_{[CLS]}$  → softmax → Level 3 at 72%
- 5 Fusion with TEWS vitals → final **Yellow** pathway

Trained on ~80,000 nurse-labeled complaints; BioBERT pre-trained on PubMed MLM.

